

EXEMPLES COMMENTES

Diagrammes UML et code Java equivalent

Neuf exemples : de la notation graphique a l'implementation

Complement au guide methodologique
base sur le support de cours de J.-P. Babau, UBO

Document de revision - M1 Informatique / ILIA

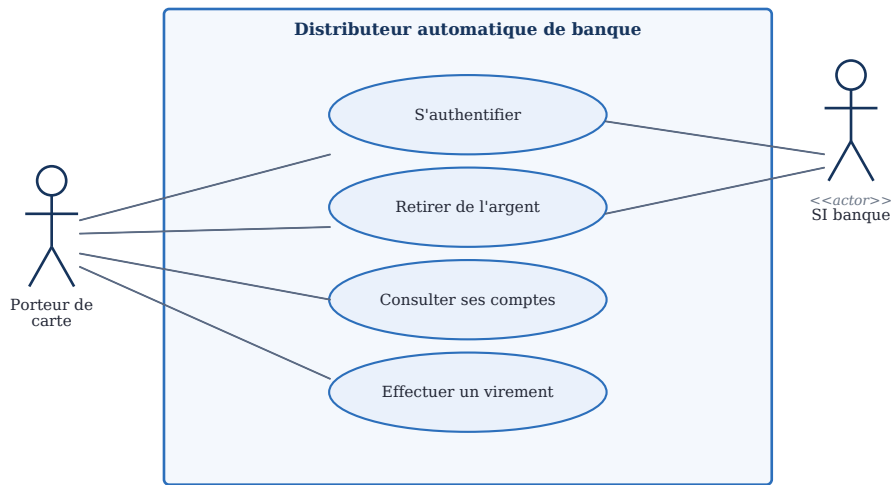
Sommaire des exemples

1. Cas d'utilisation -> interface Java du systeme
2. Heritage + association + agregation (Personne / Voiture / Roue)
3. Association simple directionnelle (Personne / Voiture)
4. Dependance (Case / Couleur)
5. Agregation seule (Voiture / Roue)
6. Composition (Echiquier / Case)
7. Interface et realisation (Comparable / Point)
8. Sequence de conception : appel de methode (Formation / Etudiant)
9. Packages (Chambres / Reservations)

Pour chaque exemple : le diagramme UML tel qu'il serait dessine, puis le code Java qui l'implemente, et une courte explication du lien entre les deux. Les regles de bonne pratique (DC, DA, DS, DP) citees sont detaillees dans le premier document.

1 Cas d'utilisation -> interface du systeme

Un diagramme de cas d'utilisation ne se code pas directement (il decrit un besoin, pas une implementation), mais chaque cas d'utilisation prefigure une **operation publique** offerte par le systeme a ses acteurs.

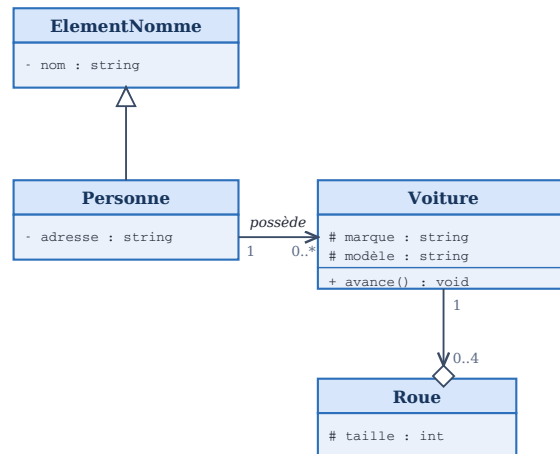


Distributeur automatique de banque

```
public interface DistributeurBancaire {  
  
    void sAuthentifier(String numeroCarte, String codeSecret);  
  
    void retirerDeLArgent(double montant);  
  
    double[] consulterSesComptes();  
  
    void effectuerUnVirement(String compteDestination, double montant);  
  
}
```

Lien diagramme -> code : chaque ellipse (cas d'utilisation) devient une methode publique ; l'acteur principal ("Porteur de carte") est celui qui appelle ces methodes ; l'acteur secondaire ("SI banque") sera sollicite a l'interieur de l'implementation (ce n'est pas lui qui appelle l'interface).

2 Heritage, association et agregation



Personne herite de ElementNomme, possede des Voiture(s), qui contiennent chacune 0 a 4 Roue(s)

```
public abstract class ElementNomme {
    private String nom;

    public String getNom() { return nom; }
    public void setNom(String nom) { this.nom = nom; }
}

public class Personne extends ElementNomme {
    protected String adresse;
    private List<Voiture> voitures = new ArrayList<>(); // 0..*

    public void ajouterVoiture(Voiture v) {
        voitures.add(v);
    }
}

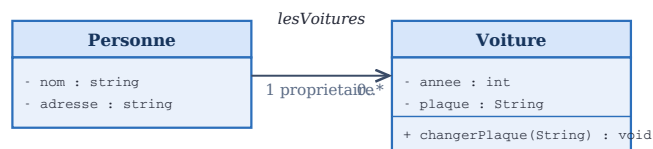
public class Voiture {
    protected String marque;
    protected String modele;
    private Roue[] roues = new Roue[4]; // agregation : 0..4 Roue

    public void avance() {
        // ...
    }
}

public class Roue {
    protected int taille;
}
```

Lien diagramme -> code : la fleche a pointe triangulaire vide devient un extends Java (**DA3** : une Personne est un cas particulier d'ElementNomme). Le losange vide (agregation) devient un simple attribut reference ou tableau -- ici, un tableau de taille 4 pour la cardinalite 0..4.

3 Association simple directionnelle



*Une Personne possède 0 à * Voiture(s) ; une Voiture appartient à un seul propriétaire*

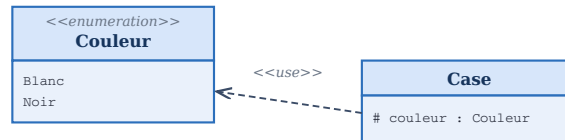
```
public class Personne {
    private String nom;
    private String adresse;
    private List<Voiture> lesVoitures = new ArrayList<>(); // 0..*
}

public class Voiture {
    private int annee;
    private String plaque;
    private Personne proprietaire; // 1

    public void changerPlaque(String nouvellePlaque) {
        this.plaque = nouvellePlaque;
    }
}
```

Lien diagramme -> code : le nom de l'association ("lesVoitures") devient directement le nom de l'attribut collection côté Personne. Chaque cardinalité se traduit par le type de l'attribut : 0..* -> une collection (List<Voiture>), 1 -> une référence simple. Règle **DA1** : en conception, on privilégie un seul sens de navigation pour réduire le couplage.

4 Dependance



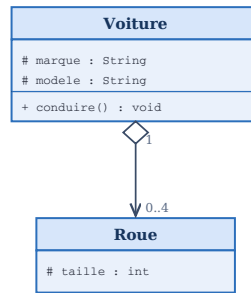
La classe Case depend de l'enumeration Couleur

```
public enum Couleur {
    BLANC,
    NOIR
}

public class Case {
    protected Couleur couleur;
}
```

Lien diagramme -> code : la fleche en tirets <<use>> correspond a l'usage d'un type dans une autre classe -- ici, comme type d'attribut. En Java, cela equivaut le plus souvent a un simple import : la modification de la classe cible (Couleur) peut impliquer une modification de la classe source (Case).

5 Agregation



Une Voiture contient 4 Roue(s) ; une Roue n'appartient qu'a une seule Voiture

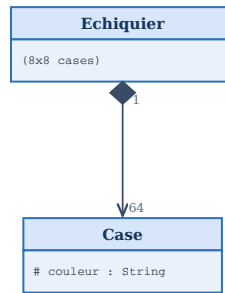
```
public class Voiture {
    protected String marque;
    protected String modele;
    private Roue[] roues = new Roue[4];    // 0..4 Roue, sans nom d'association

    public void conduire() {
        // ...
    }
}

public class Roue {
    protected int taille;
}
```

Lien diagramme -> code : le losange **vide** (agregation) devient un attribut de type reference ou tableau, mais contrairement a la composition, les objets Roue peuvent exister (et etre crees) independamment de la Voiture qui les detient.

6 Composition



Un Echiquier est compose d'exactement 64 Case(s) ; leur cycle de vie est lie a celui de l'Echiquier

```
public class Case {
    private String couleur;

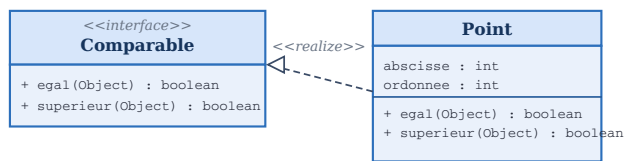
    Case(String couleur) {
        this.couleur = couleur;
    }
}

public class Echiquier {
    private final Case[] cases = new Case[64];

    public Echiquier() {
        // la creation de l'echiquier entraine la creation de ses 64 cases
        for (int i = 0; i < 64; i++) {
            cases[i] = new Case((i + i / 8) % 2 == 0 ? "Blanc" : "Noir");
        }
    }
    // aucune reference externe vers Case : sa destruction suit celle de l'Echiquier
}
```

Lien diagramme -> code : le losange **plein** (composition) se traduit par une creation des composees *a l'interieur* du constructeur du composite (regle **DA2** : l'un ne peut exister sans l'autre), et par l'absence de reference externe vers les Case -- leur duree de vie est entierement geree par Echiquier.

7 Interface et realisation



Point realise (implemente) l'interface Comparable

```
public interface Comparable {
    boolean egal(Object o);
    boolean superieur(Object o);
}

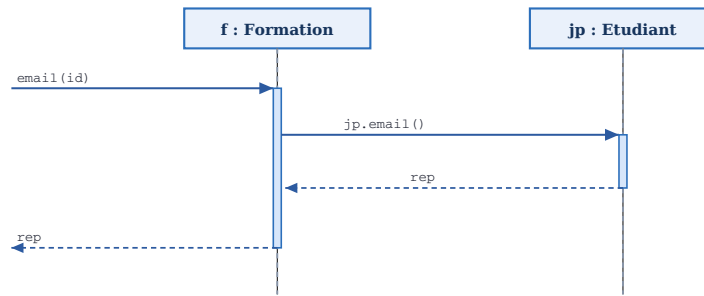
public class Point implements Comparable {
    private int abscisse;
    private int ordonnee;

    @Override
    public boolean egal(Object o) {
        Point p = (Point) o;
        return abscisse == p.abscisse && ordonnee == p.ordonnee;
    }

    @Override
    public boolean superieur(Object o) {
        Point p = (Point) o;
        return (abscisse * abscisse + ordonnee * ordonnee)
            > (p.abscisse * p.abscisse + p.ordonnee * p.ordonnee);
    }
}
```

Lien diagramme -> code : la fleche en tirets a pointe triangulaire (`<<realize>>`) devient un `implements` Java. La notion d'interface UML est directement calquee sur celle de Java : un ensemble de methodes abstraites assurant un service, sans en imposer l'implementation.

8 Sequence de conception : appel de methode



L'objet f (Formation) delegate l'appel a l'objet jp (Etudiant)

```
public class Formation {
    private String nom;
    private Map<String, Etudiant> lesInscrits = new HashMap<>();

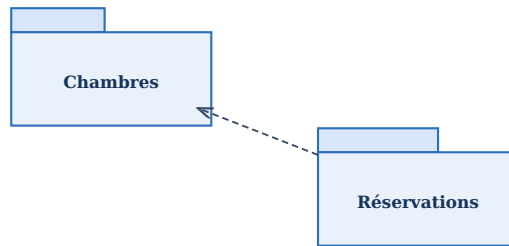
    public String email(String id) {
        Etudiant jp = lesInscrits.get(id);    // acces via l'association lesInscrits
        String rep = jp.email();              // appel de methode : le message du diagramme
        return rep;
    }
}

public class Etudiant {
    private String prenom;
    private String nom;

    public String email() {
        return prenom.toLowerCase() + "." + nom.toLowerCase() + "@etu.univ-brest.fr";
    }
}
```

Lien diagramme -> code : chaque fleche du diagramme de sequence de conception devient litteralement un appel de methode dans le code (regle **DS5**). La regle **DS6** est respectee ici : f n'envoie un message a jp que parce que jp est une variable locale obtenue via l'attribut lesInscrits.

9 Packages



Le package Reservations depend du package Chambres (jamais l'inverse)

```
// ----- package chambres -----
package hotel.chambres;

public class Chambre {
    private int numero;
    private int capacite;
}

// ----- package reservations -----
package hotel.reservations;

import hotel.chambres.Chambre;

public class Reservation {
    private Chambre chambre;    // une Reservation ne va pas sans la Chambre concernee
    private String client;
    private LocalDate dateArrivee;
}
```

Lien diagramme -> code : chaque package UML devient un package Java (package hotel.xxx;). La dependance a sens unique du diagramme se traduit par un import dans le seul sens autorisé : reservations importe des classes de chambres, jamais l'inverse (regle **DP1** : pas de dependance croisee).

Fin des exemples. Voir le premier document pour la methodologie complete de construction de chaque diagramme.